

A Geometric Index for Multi-Objective Dominance Checking

Anonymous submission

Abstract

Multi-objective A* performs dominance checks against a frontier of non-dominated cost vectors, which dominate runtime as the objective count M grows. Existing frameworks like EMOA* use linear scans or lexicographic search trees. We prove that any index using single-coordinate total order requires n dominance tests on an n -vector frontier when $M \geq 4$, whereas a geometric index answers the orthant-emptiness query in $O(n^{1-1/d})$ for $d = M - 1$. This is a genuine lower bound over antichains, holds for every leading objective, and survives randomization. We present an incremental, lazy-deletion k -d frontier answering dominance checks and set updates in place. Measured comparisons grow as n^a with $a \in [0.38, 0.84]$ across $d = 2$ to 5, tracking worst case from below. In EMOA*, our index returns bit-identical fronts while cutting dominance comparisons by $4.2\text{--}66\times$ over the lexicographic tree and $5.5\text{--}6.5\times$ over prior geometric baselines, accelerating whole-search time by up to $11\times$.

1 Introduction

Many planning and routing tasks optimize several incommensurable criteria at once: a vehicle route trades travel time against distance, monetary cost, and exposure to risk; a robot motion trades path length against clearance and energy. The Multi-Objective Shortest Path Problem (MO-SPP) models these as a graph search in which each edge carries a vector of costs, and the solution is the *Pareto front*: the set of paths that no other path beats on every objective. Unlike the single-objective case, this front can be exponentially large, and computing it exactly is the central task of multi-objective heuristic search, with applications in hazardous-materials transport, robot motion planning, and network routing (Martins 1984; Ren et al. 2025).

Because many non-dominated paths reach each vertex, multi-objective A* algorithms spend most of their time on *dominance checking*. Every newly generated path is tested against the set of non-dominated paths already found at its vertex, and the survivors update that set. As the front grows, these two operations run millions of times and come to dominate runtime. While earlier algorithms often rely on

a simple linear scan, the EMOA* framework (Ren et al. 2025) isolates these steps as the *Dominance-Check* (DC) and *Non-dominated Set Update* (NSU) problems. Like NAMOA* (Mandow and Pérez-de-la-Cruz 2010) and BOA* (Hernández et al. 2023) before it, EMOA* answers these queries using a structure keyed by the *lexicographic* order of cost vectors. This lexicographic balanced binary search tree is the state of the art for a general objective count M .

We show this choice is asymptotically suboptimal for four or more objectives ($M \geq 4$). The lexicographic tree, like any index that locates candidates through a single-coordinate total order, can confine the dominators of a query to a contiguous interval only up to three objectives. At four or more it must test every one of the n stored vectors, and this is a lower bound rather than an artifact of one implementation: for every such index we exhibit an antichain and a query on which it cannot skip a single vector without answering some indistinguishable instance incorrectly (Theorem 1). Nor is it an artifact of a bad leading objective, since it holds for every choice (Proposition 2). To escape it we use a *geometric index*, a spatial structure partitioning candidates across several coordinates at once, which answers the equivalent orthant-emptiness query in $O(n^{1-1/d})$ time for $d = M - 1$.

We use the k -d tree as our geometric index. Our contributions are: (i) an incremental, lazy-deletion k -d frontier that answers DC and NSU in place, with a soundness proof (Proposition 1); (ii) a worst-case separation between total-order and geometric indices, proved over antichains and extended to every single-coordinate order (Section 4.1), with the construction validated in code; (iii) an account of how query cost scales with frontier size and a whole-search bound explaining why speedup tracks Pareto-front size rather than graph size (Sections 4.2–4.3); and (iv) an end-to-end evaluation in which the k -d frontier returns bit-identical fronts while cutting search time by up to $11\times$ on a grid at $M=6$ and $6.65\times$ on a robot-motion instance, with a $4.2\text{--}66\times$ reduction in dominance comparisons.

2 Background

2.1 Multi-Objective Shortest Paths

Let $G = (V, E)$ be a finite directed graph in which each edge $e \in E$ carries a cost vector $\vec{c}(e) \in \mathbb{R}_{\geq 0}^M$ of M objectives. The cost of a path π is the component-wise sum

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

$\vec{g}(\pi) = \sum_{e \in \pi} \vec{c}(e)$. For two vectors $a, b \in \mathbb{R}^M$ we say a *weakly dominates* b , written $a \preceq b$, when $a_i \leq b_i$ for all $i \in \{1, \dots, M\}$, and a *dominates* b , written $a \prec b$, when in addition $a \neq b$. A path is Pareto-optimal when no other path's cost vector dominates its own. Given a start v_o and goal v_d , the Multi-Objective Shortest Path Problem (MO-SPP) asks for a maximal *cost-unique* Pareto-optimal set: one path per non-dominated cost vector reaching v_d . The number of such vectors can grow exponentially in $|V|$, which is what makes the problem hard (Martins 1984; Ren et al. 2025).

2.2 Multi-Objective A* and Lexicographic Order

Multi-objective search descends from Martins' label-correcting algorithm (Martins 1984) and Multi-Objective A* (Stewart and White 1991). NAMOA* (Mandow and Pérez-de-la-Cruz 2010) casts it as a best-first search over *labels*. A label is a path from the start to a vertex v together with its accumulated *cost-to-come* vector $\vec{g}$. Since several distinct paths can reach the same v , the search stores at each vertex the set of nondominated labels and expands them in a nondominated order.

Like A*, the search uses a heuristic $\vec{h}(v)$ estimating the *cost-to-go* from a vertex v to the goal v_d . A heuristic is *admissible* if it never overestimates any objective of any Pareto-optimal completion, and *consistent* if $\vec{h}(u) \preceq \vec{c}(u, v) + \vec{h}(v)$ for every edge (u, v) . We use a *single-valued* heuristic, returning one vector per vertex, typically the *ideal-point* heuristic $\vec{h}^{\text{ideal}}(v) = (h_1(v), \dots, h_M(v))$ whose i -th component is the optimal scalar cost-to-go for objective i alone, computed by running one backward single-source shortest-path search from the goal for each objective separately.

A recurring technique, central to what follows, is to order OPEN *lexicographically* by the f -vector $\vec{f} = \vec{g} + \vec{h}$, comparing two f -vectors on their first component and falling back to the next only on a tie. Each iteration pops the lexicographically smallest $\vec{f}$, so every label with a smaller one has already been popped. Under a consistent single-valued heuristic the offset $\vec{h}(v)$ is the same for all labels at a fixed v , so labels at v are popped in lexicographic order of $\vec{g}$. A newly arriving label at v therefore has a first component no smaller than every stored vector there, so the first-coordinate condition of the test “does a stored vector weakly dominate the new one” is redundant. Frontiers consequently store only the trailing $d = M - 1$ components and every dominance test is d -dimensional. We call this *dimensionality reduction*; NAMOA*dr (Pulido, Mandow, and Pérez-de-la-Cruz 2015), whose suffix abbreviates exactly this, exploits it when some objectives are implied by others. Note the tests use *weak* dominance, so a label whose cost equals a stored one is pruned, which yields the cost-unique set MO-SPP asks for.

At $M = 2$ the reduction is decisive, leaving $d = 1$: the frontier collapses to a single scalar, the smallest trailing component so far, and a new label is dominated exactly when its trailing component is at least that value, one comparison after an $O(\log n)$ lookup. BOA* (Hernández et al. 2023) exploits this and is the state of the art for bi-objective search. Section 4.1 shows a form of the shortcut survives at $M = 3$ but fails from $M = 4$ on.

Algorithm 1 EMOA* search (Ren et al. 2025)

```

1:  $l_o \leftarrow (v_o, \vec{0})$ ; add  $l_o$  to OPEN;  $\mathcal{F}(v) \leftarrow \emptyset \forall v$ 
2: while OPEN  $\neq \emptyset$  do
3:    $l \leftarrow \text{OPEN.pop}()$  ▷ lexicographic min  $\vec{f}$ 
4:   if IsDOMINATED( $l, \mathcal{F}(v(l))$ ) then
5:     continue ▷ DC: prune
6:   INSERT( $l, \mathcal{F}(v(l))$ ) ▷ NSU: update frontier
7:   if  $v(l) = v_d$  then
8:     continue
9:   for all  $v' \in \text{succ}(v(l))$  do
10:     $l' \leftarrow \text{extend } l \text{ to } v'$ 
11:    if not IsDOMINATED( $l', \mathcal{F}(v')$ ) then
12:      add  $l'$  to OPEN
13: return  $\mathcal{F}(v_d)$ 

```

2.3 The EMOA* Framework

The EMOA* framework (Ren et al. 2025), short for Enhanced Multi-Objective A*, unifies these algorithms and isolates their computational core. EMOA* primarily decides *when* a dominance check is performed; our contribution is orthogonal, changing *how* the per-vertex check is answered. Each vertex v maintains a *frontier* $\mathcal{F}(v)$, the cost vectors of labels reaching v that are *mutually non-dominated*. Such a set is an *antichain*: in order theory, a subset of a partially ordered set no two of whose elements are comparable, here meaning no stored vector dominates another. Every frontier is an antichain, and every frontier index must preserve that across updates. Algorithm 1 sketches the search. Two operations recur, and we keep their names throughout: the *Dominance-Check* (DC) problem IsDOMINATED($l, \mathcal{F}$) asks whether some vector in $\mathcal{F}$ weakly dominates the cost of a new label l , and the *Non-dominated Set Update* (NSU) problem INSERT($l, \mathcal{F}$) adds a survivor's cost vector and removes any stored vectors it now dominates; Algorithms 2 and 3 are our implementations, written on cost vectors directly. Both run millions of times per search and become the dominant cost as M grows (Ren et al. 2025); everything here concerns the data structure that answers them. EMOA* generalizes NAMOA*/NAMOA*dr (which keep $\mathcal{F}(v)$ in an explicit list) and BOA* (which exploits the $M = 2$ lexicographic shortcut); its reference instantiation answers DC and NSU with a balanced binary search tree keyed by lexicographic order.

2.4 k -d Trees and Orthant Queries

A dominance check is a geometric query: asking whether a cost vector g is dominated is the same as asking whether the *lower orthant* at g , the region $(-\infty, g] = \{x \in \mathbb{R}^d : x_i \leq g_i \forall i\}$, contains any stored vector. The spatial structure of non-dominated sets has been exploited before, to find the maxima of a *static* vector set (Kung, Luccio, and Preparata 1975; Bentley 1980) and to maintain nondominated archives in evolutionary optimization (Sun and Steuer 1996; Schütze 2003; Mostaghim and Teich 2005), and dominance pruning with spatial indexes has recently been studied for classical planning (Torralba and Hoffmann 2015; Salerno Garmendia, Fišer, and Torralba 2026). The gap this paper addresses is the use of a geometric index for the exact, dynamic, per-vertex

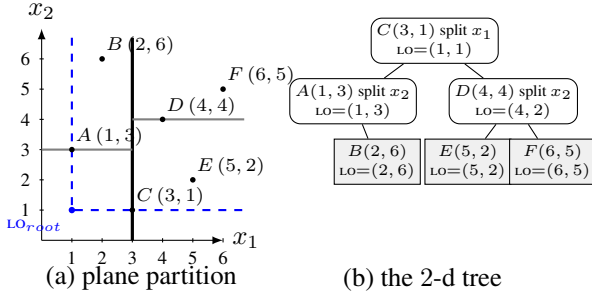


Figure 1: A 2-d tree over six points. The root C splits on x_1 and its children A and D on x_2 , the axes alternating with depth. Every node stores a point and the corner-minimum LO of its subtree; the dashed lines show the bounding box of all six points, whose near corner is LO_{root} . A query prunes a subtree when $LO \not\leq g$, so $g = (2, 0)$ prunes at the root. These six points illustrate the tree and are not a frontier, which would have to be an antichain: here $A \preceq B$ and $C \preceq E$.

frontiers of multi-objective graph search, where one structure must answer a query and absorb an update millions of times.

A k -d tree (Bentley 1975) stores d -dimensional points in a binary tree of recursive coordinate splits. A node at depth i splits on axis $a = i \bmod d$, axes numbered $0, \dots, d-1$: it stores a point whose a -th coordinate is the median along that axis among its subtree’s points, and the rest are partitioned into two children by that coordinate. Every node thus stores one point and splits its axis-aligned region (Figure 1), and median splitting gives depth $O(\log n)$. Two details matter later. A bulk build partitions at the median by selection, so points whose split coordinate *equals* the median may fall either side, whereas the incremental insertion of Section 3 sends strictly smaller values left and equal or larger right; and our query never consults a split coordinate at all, pruning on the corner annotation and otherwise descending into both children, so tie placement affects balance, not correctness.

Annotating every node ν with the *corner-minimum* $LO(\nu)$, the vector whose i -th entry is the smallest i -th coordinate among the subtree’s points, turns a dominance test into an *orthant-emptiness* query: does any stored point lie in $(-\infty, g]$? A subtree may be skipped whenever $LO(\nu) \not\leq g$, since every point x in it then has $x_i \geq LO(\nu)_i > g_i$ on the offending axis and cannot dominate g . Geometrically $LO(\nu)$ is the near corner of the bounding box of those points, and the test asks whether that box meets the query orthant; it is at least as strong as the classical test on the node’s split region (Appendix A), so the $O(dn^{1-1/d})$ bound for orthant queries on a balanced tree in $O(n)$ space (Lee and Wong 1977) still applies. Building takes $O(n \log n)$. In Figure 1 the root stores $C(3, 1)$, splits on $x_1=3$, and carries $LO=(1, 1)$; querying with $g=(2, 0)$ gives $LO \not\leq g$ because $1 > 0$ in the second coordinate, pruning the whole tree without testing a point.

3 Geometric Frontier Index

A k -d tree with corner-minima can prune orthant-emptiness queries, but it is built once over a static point set and offers no deletion, which a frontier needs: inserting a new non-

dominated vector can make stored members dominated, so they must leave the set. Removing them eagerly requires restructuring the affected subtree, which is computationally expensive. We avoid that with *dead nodes*: a dominated point is marked, stays in the tree to route insertions, is skipped when testing dominance, and is removed in bulk later. This yields a dynamic k -d tree answering DC and NSU in place.

Index Node. Each node ν stores a cost vector $POINT(\nu)$, a live/dead bit, a split axis, two child pointers, and two corners: the componentwise minimum $LO(\nu)$ and maximum $HI(\nu)$ over all points in ν ’s subtree, dead ones included. It keeps three counters, all zero initially: the physical node count *nodes*, the live count *live*, and *live_{last}*, a snapshot of *live* taken at the last rebuild. A node is *live* when its point is a current frontier member and *dead* when a later insertion dominated it but it remains present.

Check (DC). Algorithm 2 answers *IsDOMINATED* by depth-first search, pruning any subtree whose corner-minimum cannot dominate the query, since $LO(\nu)_i > g_i$ for some axis i implies no point below is $\preceq g$, and returning *TRUE* on the first live dominator. Dead nodes are skipped in the point test but their children are still traversed.

Update (NSU). Algorithm 3 inserts a new non-dominated g in two passes. *MARKDOMINATED* is the symmetric *upper-orthant* sweep: it visits $[g, \infty)$ using the corner-maxima HI and marks each live p with $g \preceq p$ dead, decrementing *live*. Then *INSERTNODE* walks to a leaf along the split axes, attaches a live node for g , and refreshes both corners on the path in $O(\text{depth})$ time.

Reclaiming dead nodes. A bulk rebuild periodically discards the dead nodes and rebuilds a balanced tree from the live points, splitting each level at the median of its axis (found in linear time by selection). It triggers only when *nodes* exceeds $2 \text{live}_{last} + s$, for a small constant slack s that suppresses rebuilds on tiny frontiers ($s=8$ throughout). Since a rebuild sets *nodes* = *live* = *live_{last}*, the tree holds at most $2 \text{live}_{last} + s + 1$ nodes and so stays $O(n)$, and reaching the trigger takes at least *live_{last}* + $s + 1$ insertions, so the $O(m \log m)$ rebuild over m live points amortizes to $O(\log n)$ per update. Comparing against the snapshot rather than the current live count bounds physical size, which is what Section 4.3 uses, rather than the fraction of dead nodes; Appendix E gives the variant that bounds both. Appendix D traces the operations.

Rebuilds also restore depth, though between them an adversarial insertion order can degrade it, so the $O(dn^{1-1/d})$ bound of Section 4.1 is guaranteed only for a balanced dynamization (Cai, Xu, and Zhang 2021; Brown 2025; Bentley and Saxe 1980). Experiment 1 measures what that costs, by replaying the same streams through a variant that keeps the tree balanced at all times.

Proposition 1 (Lazy-deletion soundness). *Consider the tree after any sequence of INSERT operations and rebuilds, and let L be its set of live points. For every query q , CHECK returns TRUE if and only if some $p \in L$ satisfies $p \preceq q$. Retaining dead nodes therefore changes neither the result of a CHECK nor the frontier it represents.*

The proof (Appendix A) turns on an invariant that the two-pass structure of Algorithm 3 makes non-obvious: every dead

Algorithm 2 $\text{CHECK}(\nu, g)$: the DC operation IsDOMINATED on the k -d frontier

Require: node ν and query $g \in \mathbb{R}^d$; call as $\text{CHECK}(\text{root}, g)$
Ensure: TRUE if and only if some live stored point weakly dominates g

```

1: if  $\nu = \text{NIL}$  then
2:   return FALSE
3: if  $\text{LO}(\nu)_i > g_i$  for some axis  $i$  then
4:   return FALSE ▷ corner prune
5: if  $\nu$  is live and  $\text{POINT}(\nu) \preceq g$  then
6:   return TRUE
7: return  $\text{CHECK}(\text{left}(\nu), g) \vee \text{CHECK}(\text{right}(\nu), g)$ 

```

Algorithm 3 $\text{INSERT}(g)$: the NSU operation on the k -d frontier

Require: new vector $g \in \mathbb{R}^d$ for which CHECK returned FALSE

```

1:  $\text{MARKDOMINATED}(\text{root}, g)$  ▷ mark dominated as dead
2:  $\text{root} \leftarrow \text{INSERTNODE}(\text{root}, g, 0)$ 
3:  $\text{nodes} += 1$ ;  $\text{live} += 1$ 
4: if  $\text{nodes} > 2 \cdot \text{live}_{\text{last}} + s$  then
5:   rebuild a balanced  $k$ -d tree from the live points
6:    $\text{nodes} \leftarrow \text{live}$ ;  $\text{live}_{\text{last}} \leftarrow \text{live}$ 
7: function  $\text{MARKDOMINATED}(\nu, g)$ :
8:   if  $\nu = \text{NIL}$  then
9:     return
10:  if  $\text{HI}(\nu)_i < g_i$  for some axis  $i$  then
11:    return ▷ no point  $\succeq g$  below
12:  if  $\nu$  is live and  $g \preceq \text{POINT}(\nu)$  then
13:    mark  $\nu$  dead;  $\text{live} -= 1$ 
14:   $\text{MARKDOMINATED}(\text{left}(\nu), g)$ ;
     $\text{MARKDOMINATED}(\text{right}(\nu), g)$ 
15: function  $\text{INSERTNODE}(\nu, g, a)$ :
16:   if  $\nu = \text{NIL}$  then
17:     return live leaf  $(g)$  with axis  $a$ 
18:    $\text{LO}(\nu) \leftarrow \min(\text{LO}(\nu), g)$ ,  $\text{HI}(\nu) \leftarrow \max(\text{HI}(\nu), g)$ 
19:   if  $g_a < \text{POINT}(\nu)_a$  then
20:      $\text{left}(\nu) \leftarrow \text{INSERTNODE}(\text{left}(\nu), g, (a+1) \bmod d)$ 
21:   else
22:      $\text{right}(\nu) \leftarrow \text{INSERTNODE}(\text{right}(\nu), g, (a+1) \bmod d)$ 
23:   return  $\nu$ 

```

point is dominated by a *live* one. A dead point’s original killer may itself be killed later, so the witness must be chased forward through the kill chain, which an induction over insertions does. The corners need a separate argument, since dead points do enter LO and HI ; there the effect is on cost, not correctness, because including them can only prune *less*. Consistent with this, every backend in Section 5 is checked against a brute-force linear oracle and the k -d frontier returns bit-identical fronts on every instance.

4 Analysis

This section analyzes the cost of DC and NSU as a function of n , the number of live cost vectors at a vertex. Those n vectors always form an antichain, and we work throughout in the

dimensionality-reduced space of $d = M - 1$ coordinates of Section 2: because labels are popped in lexicographic order, a new label’s first component is no smaller than any stored one, so the first-coordinate condition of every dominance test holds before the test begins and cannot discriminate. All comparisons run on the remaining d coordinates, and n , d and the frontier refer to that reduced space. We bound the worst case and prove it separates total-order from geometric indices (Section 4.1), ask what a query costs on the frontiers a search actually builds (Section 4.2), then aggregate over a whole search (Section 4.3).

4.1 Worst-Case Cost and a Separation

Upper bounds. On a k -d tree over n points in d dimensions a CHECK is an orthant-emptiness query costing $O(d n^{1-1/d})$ time in $O(n)$ space (Lee and Wong 1977). The fractional exponent arises because the boundary of a query orthant is a $(d - 1)$ -dimensional surface, and in the worst case only the nodes whose regions that boundary cuts must be inspected; over a d -dimensional partition of n points, the number of cut regions scales as $O(n^{(d-1)/d})$. Returning on the first dominator only lowers this. An INSERT inserts in $O(\log n)$, marks the dominated orthant by a second traversal, and amortizes its rebuild to $O(\log n)$ by the doubling rule of Section 3.

High-dimensional limit. As d grows, $1 - 1/d \rightarrow 1$ and the d prefactor rises, so the worst-case query approaches the linear scan’s $O(n)$; the curse of dimensionality bounds where the geometric advantage plateaus. For MO-SPP the reduced dimension $d = M - 1$ is small ($d \leq 5$ here), so the plateau lies beyond the practical range, and the measured cost is far below this worst case (Section 4.2), which is why the advantage keeps growing with M . Many more objectives would need a complementary technique that we do not address.

Definition 1 (Total-order and coordinate-lexicographic indices). A total-order index for a fixed linear order $\prec$ on cost vectors answers CHECK in two stages. It first selects a contiguous $\prec$ -interval of the stored vectors, using only $\prec$ -comparisons between the query and stored vectors, and then dominance-tests the vectors in that interval and reports whether any dominates the query. We charge only the dominance tests, so selection is free. The index is correct on a class of inputs when it reports a dominator whenever one exists, for every stored set in the class and every query. It is a coordinate-lexicographic index when $\prec$ is the lexicographic order. The balanced binary search tree and binary-search list of Ren et al. (2025) are of this form: both bound the leading coordinates to locate candidates, then test them. Charging nothing for selection only strengthens the bound below, which then holds even for an index whose $O(\log n)$ search is free.

Theorem 1 (Separation). Let the stored set be an antichain of n cost vectors in $\mathbb{R}^d$, a set in which no vector dominates another. (i) For $d = 2$ some coordinate-lexicographic index is correct on all antichains and answers every CHECK with $O(1)$ dominance tests. (ii) For every $d \geq 3$, every $n \geq 1$ and every total-order index correct on all n -point antichains in $\mathbb{R}^d$, there are an n -point antichain P and a query g on which that index performs exactly n dominance tests. (iii) On those instances a k -d frontier answers CHECK with 0 dominance

tests after one corner test, and on any balanced k -d tree over n points it uses $O(d n^{1-1/d})$ (Lee and Wong 1977).

Proof sketch for (ii); full proofs in Appendix A, and Figure 3 there draws the instance. Fix $d \geq 3$ and $n \geq 1$ and take the staircase instance

$$P = \{p^{(1)}, \dots, p^{(n)}\}, \quad p^{(i)} = (i, n+1-i, 1, \dots, 1),$$

with query $g = (n, n, 1, \dots, 1, 0)$. Three facts combine. First, the descending first two coordinates make any two points incomparable *whatever* their remaining coordinates are, so altering one point's last coordinate leaves an antichain; this is where $d \geq 3$ enters, two coordinates spent on incomparability and a third left over to hide dominance in. Second, no point dominates g , since every $p_d = 1 > 0 = g_d$, yet $p_i \leq g_i$ for all $i < d$. Third, if the index omits some $p^{(k)}$, lowering $p_d^{(k)}$ to 0 gives another antichain in which that point dominates g , and no $\prec$ -comparison reads coordinate d : the distinct first coordinates settle every comparison between stored vectors, and coordinate 1 or 2 every comparison against g . The index behaves identically on the modified instance, omits $p^{(k)}$ again, and answers “no dominator” when one exists, so a correct index omits nothing. Part (i) is the two-dimensional staircase argument making $p_2^{(j)} \leq g_2$ a single decisive test; part (iii) checks that corner pruning dominates the classical split-plane test, so the bound of Lee and Wong (1977) transfers. Because the instances are mutually indistinguishable rather than chosen after the index commits, the bound also survives randomization (Corollary 1).

Scope of the lower bound. Nothing in the proof is specific to the lexicographic order. It extends to every *single-coordinate* order, meaning order by one chosen objective and break ties lexicographically, a family covering the indices used by the state of the art (Ren et al. 2025; Mandow and Pérez-de-la-Cruz 2010; Hernández et al. 2023).

Fix any objective j and let $\prec_j$ order cost vectors by coordinate j with ties broken lexicographically. Proposition 2 (Appendix A) shows that for $d \geq 3$ and every n , any index for $\prec_j$ correct on n -point antichains again performs exactly n dominance tests on some instance. Its proof runs the staircase in coordinate j and one other, hides dominance in a third, and puts the query's j -th coordinate above every stored value, so coordinate j decides every comparison and the tie-break never runs. Within this family the picture is complete: no choice of objective escapes the n -test worst case, while a geometric index is sufficient at $O(n^{1-1/d})$. Table 1 shows the crossover: the two agree within a constant factor at $M=3$, but at $M=4$ the lexicographic tree does 92 comparisons per check against the k -d tree's 3.3.

4.2 Average-Case Cost

The construction in Section 4.1 establishes what a query can cost in the worst case, but does not predict costs on search-generated frontiers. We use two arguments to motivate an empirical hypothesis; Appendix B gives the assumptions and limitations in full. First, suppose the reduced cost vectors of the r labels reaching a vertex are independent draws from a continuous distribution with independent components. The

frontier is then the set of sample minima, whose expected size is $\mathbb{E}[n] \sim (\ln r)^{d-1}/(d-1)!$ for fixed d (Bentley et al. 1978). Under this model, the frontier size n therefore grows polylogarithmically, rather than linearly, with r . This model excludes dependence between objectives; positive correlation tends to align them and leave fewer surviving trade-offs.

Second, the corner test can prune an entire subtree whenever its coordinatewise minimum does not dominate the query. A dispersed antichain should make such pruning common, but no available average-case theorem covers this setting. The closest result analyses partial-match queries on an order-independent random sample (Flajolet and Puech 1986); our queries are orthant queries, and search-generated labels have correlated values and a structured insertion order. We therefore claim no average-case bound. Instead, Experiment 2 tests whether comparisons per CHECK follow n^a for an exponent $a < 1$, and compares the fitted exponent with the worst-case exponent $1 - 1/d$.

4.3 Whole-Search Behavior

The previous subsections bound one operation at one vertex; this one aggregates them. Let $r(v)$ be the number of labels generated at vertex v , $n(v)$ the largest frontier size reached there, and $\kappa(n)$ an upper bound on the dominance comparisons a CHECK costs on a frontier of n vectors. Algorithm 1 performs at most two CHECK calls per label, on pop and on generation, so

$$W_{\text{chk}} = \sum_v \sum_{\text{labels at } v} (\text{CHECK cost}) \leq 2 \sum_v r(v) \kappa(n(v)),$$

where $n(v)$ may replace the frontier size at each call because κ is non-decreasing. Here $\kappa(n) = n$ for the linear scan and $O(d n^{1-1/d})$ for a balanced k -d tree, with the measured exponent below $1 - 1/d$ (Experiment 2); update work adds a term of the same shape over the labels that survive their check, and we report the two separately throughout.

The consequence is what we test: backends differ only through κ , so speedup depends on the distribution of frontier sizes encountered across vertices, $\{n(v)\}$, not the vertex count $|V|$. Similar fronts should give similar speedups despite graph size, while a large graph with small fronts should give little gain. Experiment 3 confirms the first and Experiment 4 the second, refining the dominance-check component of the analysis of Ren et al. (2025).

5 Experiments

Setup. Domains. *Synthetic instances* are 30×30 four-neighbor grids whose directed edges carry M integer costs from a correlated generator: each edge draws a base cost b uniformly from the integers $[1, 20]$ as its first objective, and each remaining objective is $\lfloor \rho b + (1 - \rho)u \rfloor$ with u an independent draw from the same range and $\lfloor \cdot \rfloor$ rounding to the nearest integer, so $\rho \in [0, 1]$ tunes how strongly the objectives agree and hence the front size. We use $M=3-6$ and $\rho \in \{0.5, \dots, 0.9\}$. *Real-world instances* are the DIMACS road networks of Experiment 4 and a 7-DOF (degrees of freedom) Panda robot-arm road-map (1000 configurations; path length plus per-link joint-clearance objectives) from

the benchmark suite of Peer et al. (2026). All generators are seeded and deterministic. **Protocols.** The *replay* protocol isolates backend speed by recording the CHECK/UPDATE stream from the busiest vertex of a real search and replaying it through each backend; the *end-to-end* protocol runs the full search, swapping only the backend via a runtime flag. Every backend is validated against a brute-force linear oracle, and our k -d tree returns fronts bit-identical to the AVL on every instance. **Baselines.** Four backends behind one interface: a *Linear* scan; the EMOA* *AVL*, a lexicographic balanced tree; the incremental *ND-tree* (Jaszkievicz and Lust 2018), the closest prior geometric structure, which stores the frontier as axis-aligned bounding boxes and skips a subtree when its box cannot hold a dominator, without worst-case bounds; and our k -d tree. **Metrics.** Comparison counts are exact and deterministic; wall-clock is a mean over repeated runs with a 95% confidence interval. Each end-to-end speedup is confirmed by a one-sample t -test on the per-run AVL/ k -d ratio (H_0 : ratio = 1), rejected at $p < 0.001$ everywhere. Compiler, allocator and hardware details are in Appendix E.

Experiment 1: the query phase and the separation.

Question. Does the total-order index degrade at $M \geq 4$ as Theorem 1 predicts, and does the k -d tree avoid it? *Results.* Replaying the theorem’s construction reproduces both bounds exactly: on the staircase of Theorem 1(ii) at $n = 256, \dots, 16,384$ and every $d \in \{3, 4, 5, 6\}$, the AVL performs exactly n comparisons per CHECK and the k -d tree 0, pruning at the root on one corner test; the family of Remark 1 gives the same counts. On the captured streams (Table 1), where checks are $\sim 99\%$ of operations, the lexicographic tree grows from 4.3 comparisons per check at $M=3$ to 731 at $M=6$ while the k -d tree stays between 0.76 and 11.4. *Analysis.* The total-order indices grow with M as the lower bound predicts, giving the k -d tree a comparison reduction rising from $5.5\times$ at $M=3$ to $64\times$ at $M=5$ –6 and $26\times$ lower wall-clock at $M=6$ (0.75 vs. 20 s). At $M=3$ ($d=2$) all indices agree within a constant factor, the regime Theorem 1(i) makes optimal for a total order, which is the crossover the theory predicts, though optimality there is asymptotic and the constant favors the corner test (0.76 vs. 4.3). The ND-tree sits between the two at 5.5 – $6.5\times$ the k -d tree’s comparisons, running 1.9 – $2.8\times$ slower at $M=4$ –6 but faster at $M=3$, where the frontier is too small for the savings to repay the corner storage. The AVL is *slower* than the linear scan at every M despite doing 2 – $2.5\times$ fewer comparisons, its pointer chasing and rotations outweighing the savings, as Ren et al. (2025) also report; the k -d index wins on both metrics from $M=4$ up.

Does balance matter? The $O(dn^{1-1/d})$ bound is guaranteed only for a balanced tree, which the incremental form does not keep between rebuilds. Replaying the same streams through a variant that stays median-balanced at every step separates two effects (Appendix E). Pruning is nearly indifferent to balance: the balanced variant needs 1.05 – $1.53\times$ more comparisons per CHECK than the incremental one, and still does 3.6 – $6.9\times$ fewer than the ND-tree and 3.7 – $84\times$ fewer than the AVL, so the separation does not depend on which variant is used. Update time is where balance is ex-

M	Backend	t (ms)	cmp _{chk}	cmp _{upd}	cmp/chk
3	Linear	7.5 ± 0.3	280,882	3340	10.3
3	AVL	9.5 ± 0.9	118,111	2841	4.3
3	ND-tree	7.2 ± 0.5	113,045	2299	4.2
3	k -d (ours)	8.1 ± 0.9	20,657	435	0.76
4	Linear	612 ± 17	43,125,842	366,633	181
4	AVL	701 ± 18	21,921,512	183,392	92
4	ND-tree	182 ± 8	4,938,430	81,077	21
4	k -d (ours)	98 ± 2	784,288	12,028	3.3
5	Linear	5351 ± 110	371,733,274	3,814,498	695
5	AVL	8115 ± 1850	190,150,426	1,752,175	355
5	ND-tree	761 ± 24	16,456,457	432,808	31
5	k -d (ours)	380 ± 12	2,964,273	57,296	5.5
6	Linear	16789 ± 760	1,117,400,245	13,316,600	1847
6	AVL	19960 ± 440	442,000,662	7,285,564	731
6	ND-tree	2132 ± 30	44,656,691	984,202	74
6	k -d (ours)	753 ± 24	6,928,325	122,501	11.4

Table 1: **Replay of captured search streams** on the synthetic grid, one stream per objective count M , replayed identically through all four backends. t is wall-clock, a mean over K replays of the fixed stream ($K=10$ for $M=3, 4$, $K=5$ for $M=5, 6$) with a 95% confidence interval; cmp_{chk} and cmp_{upd} are total comparisons in CHECK and UPDATE, exact and so without intervals; cmp/chk is cmp_{chk} per CHECK call. Bold marks the best per block, and for wall-clock every value overlapping it. The $M=5$ and $M=6$ streams are time-capped search prefixes, as in the source benchmark, so they are shorter than the full captures replayed in Table 5.

pensive: maintaining it by rebuilding costs $O(m \log m)$ per update and leaves the balanced variant slower in wall-clock than *every* baseline at $M \geq 5$. Lazy deletion is therefore what converts the pruning advantage into a runtime one.

Experiment 2: how query cost scales with frontier size.

Question. Does the geometric index’s cost grow as a sub-linear power of n , and how does the measured exponent compare with the $n^{1-1/d}$ worst case (Section 4.2)? *Setup.* Sweeping $d \in \{2, 3, 4, 5\}$, $\rho \in \{0, 0.3, 0.5, 0.8\}$, a target frontier size in $\{20, 60, 150, 400\}$ and three seeds gives 48 streams of 20,000 operations per dimension; we fit $\log(\text{comparisons per CHECK})$ against $\log n$ *separately for each d* , since pooling would be misleading when the predicted exponent $1 - 1/d$ itself depends on d . *Results.* The k -d tree’s exponent rises with dimension as predicted and stays close to, and mostly below, the worst-case $1 - 1/d$ (Figure 2): $a = 0.38 \pm 0.01$ at $d=2$ (worst case 0.50), 0.55 ± 0.02 at $d=3$ (0.67), 0.74 ± 0.02 at $d=4$ (0.75) and 0.84 ± 0.02 at $d=5$ (0.80), with $R^2 \geq 0.95$ and intervals of ± 1 standard error. Absolute cost is far below what the bound allows: at $d=4$, $n=6878$, it does 22.9 comparisons per CHECK against $n^{1-1/d} = 755$. *Analysis.* The total-order indices fit slopes above 1 at every dimension (1.21–1.82 linear, 1.22–1.78 AVL). These are empirical slopes over the measured range, not growth rates: a scan costs at most n comparisons and stayed below n in all 192 runs. The slope exceeds 1 only because the scanned fraction grows with n , from 0.02–0.22 to 0.43–0.50. The k -d tree’s sub-linear exponent, by contrast, reflects real pruning.

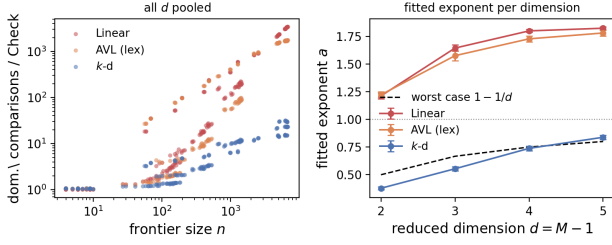


Figure 2: Scaling of dominance comparisons per CHECK with frontier size n , over 48 streams per reduced dimension $d = M - 1$. *Left*: raw measurements, all d pooled, log-log axes. *Right*: the exponent a from fitting comparisons per CHECK $\propto n^a$ separately at each d , with ± 1 standard error, against the $1 - 1/d$ worst case (dashed) and the linear-cost line $a=1$ (dotted). The k -d tree tracks the worst-case exponent from below; the total-order indices sit above $a=1$ only because the fraction of the frontier they scan grows over the measured range, which cannot continue since a scan costs at most n .

M	Sol.	AVL (s)	k -d (s)	Speedup	chk↓
<i>Synthetic grid</i>					
3	209	0.061 ± 0.002	0.056 ± 0.001	1.10 ± 0.04	4.2
4	1776	1.250 ± 0.019	0.695 ± 0.010	1.80 ± 0.03	15.9
5	9140	29.98 ± 1.54	4.21 ± 0.17	7.12 ± 0.33	54.8
6	13,705	86.49 ± 1.43	7.82 ± 0.06	11.06 ± 0.25	66.1
<i>Robot arm (7-DOF)</i>					
5	14,766	216.4 ± 6.9	32.55 ± 0.25	6.65 ± 0.17	28.3

Table 2: **End-to-end search time** on the synthetic grid ($M=3-6$) and a real 7-DOF robot-arm instance. Sol. is the number of cost-unique Pareto-optimal solutions, i.e., the goal frontier size; AVL and k -d are mean wall-clock times with a 95% confidence interval over K full searches ($K = 25, 15, 5, 4$ for $M=3, \dots, 6$ and $K = 3$ for the arm). Speedup is the mean of per-run ratios; chk↓ is the AVL/ k -d ratio of total CHECK comparisons, exact. Every interval lies above 1, and the fronts are bit-identical throughout.

Experiment 3: end-to-end speedup and the front-size law.

Question. Does the win survive in a full search, and grow with the Pareto-front size as Section 4.3 predicts? *Results.* In Table 2 every speedup interval lies above 1 and the fronts are bit-identical. On the grid, where the graph is fixed at 900 vertices and only M varies, the speedup rises monotonically with the front, from 1.10 at 209 solutions to 11.06 at 13,705, and the comparison reduction with it, from $4.2\times$ to $66\times$; on the Panda arm ($M=5$ active, 14,766 solutions) it is $6.65\times$ with a $28.3\times$ reduction. *Analysis.* Fixing the graph and varying only the front isolates the mechanism: the backends differ only in κ , so the gain must track frontier size, and it does over two orders of magnitude. It grows more slowly than the comparison reduction ($11\times$ against $66\times$), since comparisons are only part of a search that also pays for OPEN operations and label allocation.

Experiment 4: real road networks via replay at scale.

Question. Do the gains hold on real topologies with realistic costs, across maps, objective counts and correlations, and how do coherent and random costs differ? *Setup.* Three DIMACS road networks (New York, 264,346 vertices; San Francisco Bay, 321,270; Colorado, 435,666). Their distance and time weights give only two near-perfectly correlated objectives and hence tiny fronts, so we synthesize objectives with the disclosed *landmark-field* model (Appendix C), sweeping $M \in \{4, 5\}$ and $\rho \in \{0.5, \dots, 0.7\}$ against a random i.i.d. control. On the largest fronts a full search overflows any budget and the backends return *different* truncated fronts, so we replay.

Results. Figure 5 and Table 4 report over 19 captured streams the AVL/ k -d reduction in check comparisons, update comparisons and replay wall-clock against the per-vertex frontier size n , which ranges from 4 to 678. Under geometric costs the check reduction spans $4.1-46.9\times$ and the update reduction $1.6-22.3\times$, while wall-clock rises from parity at $n=4$ to $7.4\times$ at $n=418$. Random i.i.d. costs give $104\times$ in comparisons and $12.9\times$ in wall-clock at $n=578$, so coherent real costs are the *harder* case and the geometric numbers are the conservative ones.

Analysis. The reductions grow with frontier size rather than graph size: single- and double-digit fronts cluster between $4\times$ and $9\times$ whichever map they come from, while both largest fronts reach $19\times$. The trend is not monotone, two Bay $M=4$ streams reaching $32\times$ and $46.9\times$ at $n=14$ and 66 where the k -d tree’s cost per check is lowest, so M and ρ modulate it alongside n . Against the ND-tree the k -d tree does $1.8-6.5\times$ fewer check and $2.3-11.8\times$ fewer update comparisons, though at small n the wall-clock difference is within noise. As in Experiment 3, counts and time diverge sub-proportionally: $104\times$ fewer comparisons yields $12.9\times$ in wall-clock.

Experiment 5: sensitivity to objective ordering. Salzman et al. (2023) leave open which objective should be tested first, and Proposition 2 says no choice removes the worst case. Replaying every stream under all $d!$ orderings (Appendix F) gives the practical cost: the total-order baselines swing by up to $4.7\times$ (AVL) and $12.1\times$ (sorted linear) in comparisons per check, the k -d index by at most $1.82\times$, whose *worst* ordering still beats every baseline’s *best*.

6 Conclusion

Dominance checking governs the cost of multi-objective A*. We proved that a lexicographic index is optimal at $M=3$ but must test all n stored vectors at $M \geq 4$, that no choice of leading objective escapes this, and that a geometric index is sufficient at $O(n^{1-1/d})$; a whole-search bound explains why the speedup tracks Pareto-front size rather than graph size, and the evaluation gives bit-identical fronts with up to $11\times$ faster search. The bound extends to ε -approximate search, where each test is an orthant query against an ε -shifted point. Three questions remain open: a lower bound for orders outside Definition 1, an average-case bound on search-generated frontiers, and multi-valued heuristics, whose batched queries could share corner annotations.

References

- Bentley, J. L. 1975. Multidimensional Binary Search Trees Used for Associative Searching. *Commun. ACM*, 18(9): 509–517.
- Bentley, J. L. 1980. Multidimensional Divide-and-Conquer. *Commun. ACM*, 23(4): 214–229.
- Bentley, J. L.; Kung, H. T.; Schkolnick, M.; and Thompson, C. D. 1978. On the Average Number of Maxima in a Set of Vectors and Applications. *J. ACM*, 25(4): 536–543.
- Bentley, J. L.; and Saxe, J. B. 1980. Decomposable Searching Problems I: Static-to-Dynamic Transformation. *J. Algorithms*, 1(4): 301–358.
- Brown, R. A. 2025. A Dynamic, Self-balancing k-d Tree. *CoRR*, abs/2509.08148.
- Cai, Y.; Xu, W.; and Zhang, F. 2021. ikd-Tree: An Incremental K-D Tree for Robotic Applications. *CoRR*, abs/2102.10808.
- Flajolet, P.; and Puech, C. 1986. Partial match retrieval of multidimensional data. *J. ACM*, 33(2): 371–407.
- Hernández, C.; Yeoh, W.; Baier, J. A.; Zhang, H.; Suazo, L.; Koenig, S.; and Salzman, O. 2023. Simple and efficient bi-objective search algorithms via fast dominance checks. *Artif. Intell.*, 314: 103807.
- Jaszkiewicz, A.; and Lust, T. 2018. ND-Tree-Based Update: A Fast Algorithm for the Dynamic Nondominance Problem. *IEEE Trans. Evol. Comput.*, 22(5): 778–791.
- Kung, H. T.; Luccio, F.; and Preparata, F. P. 1975. On Finding the Maxima of a Set of Vectors. *J. ACM*, 22(4): 469–476.
- Lee, D. T.; and Wong, C. K. 1977. Worst-Case Analysis for Region and Partial Region Searches in Multidimensional Binary Search Trees and Balanced Quad Trees. *Acta Informatica*, 9: 23–29.
- Madow, L.; and Pérez-de-la-Cruz, J. 2010. Multiobjective A* search with consistent heuristics. *J. ACM*, 57(5): 27:1–27:25.
- Martins, E. Q. V. 1984. On a Multicriteria Shortest Path Problem. *European Journal of Operational Research*, 16(2): 236–245.
- Mostaghim, S.; and Teich, J. 2005. Quad-trees: A Data Structure for Storing Pareto Sets in Multiobjective Evolutionary Algorithms with Elitism. In Abraham, A.; Jain, L. C.; and Goldberg, R. R., eds., *Evolutionary Multiobjective Optimization*, Advanced Information and Knowledge Processing, 81–104. Springer.
- Peer, H.; Hernández, C.; Koenig, S.; Felner, A.; and Salzman, O. 2026. Bridging the Evaluation Gap: Standardized Benchmarks for Multi-Objective Search. *CoRR*, abs/2603.24084.
- Pulido, F. J.; Madow, L.; and Pérez-de-la-Cruz, J. 2015. Dimensionality reduction in multiobjective shortest path search. *Comput. Oper. Res.*, 64: 60–70.
- Ren, Z.; Hernández, C.; Likhachev, M.; Felner, A.; Koenig, S.; Salzman, O.; Rathinam, S.; and Choset, H. 2025. EMOA*: A framework for search-based multi-objective path planning. *Artif. Intell.*, 339: 104260.
- Salerno Garmendia, M. F.; Fišer, D.; and Torralba, Á. 2026. Finding a Dominating State in a Haystack: Efficient Data Structures for Dominance Pruning. In *Proc. Int. Conf. on Automated Planning and Scheduling (ICAPS)*.
- Salzman, O.; Felner, A.; Hernández, C.; Zhang, H.; Chan, S.; and Koenig, S. 2023. Heuristic-Search Approaches for the Multi-Objective Shortest-Path Problem: Progress and Research Opportunities. In *IJCAI*, 6759–6768. ijcai.org.
- Schütze, O. 2003. A New Data Structure for the Nondominance Problem in Multi-objective Optimization. In *EMO*, 509–518.
- Stewart, B. S.; and White, C. C., III. 1991. Multiobjective A*. *J. ACM*, 38(4): 775–814.
- Sun, M.; and Steuer, R. E. 1996. Quad-Trees and Linear Lists for Identifying Nondominated Criterion Vectors. *INFORMS J. Comput.*, 8(4): 367–375.
- Torralba, Á.; and Hoffmann, J. 2015. Simulation-Based Admissible Dominance Pruning. In Yang, Q.; and Wooldridge, M. J., eds., *Proceedings of the Twenty-Fourth International Joint Conference on Artificial Intelligence, IJCAI 2015, Buenos Aires, Argentina, July 25-31, 2015*, 1689–1695. AAAI Press.

Supplementary Material

A Geometric Index for Multi-Objective Dominance Checking

A Deferred Proofs

Proposition 1 (lazy-deletion soundness). *Consider the tree after any sequence of INSERT operations and rebuilds, and let L be its set of live points. For every query q , CHECK returns TRUE if and only if some $p \in L$ satisfies $p \preceq q$.*

Proof. *Step 1: every dead point is dominated by a live point.* We claim that at the end of every INSERT, each dead point p admits a live $\hat{g}$ with $\hat{g} \preceq p$, by induction on the number of completed insertions. Initially there are no dead points. For the step, consider the insertion of g . Its MARKDOMINATED pass marks a point q dead only when $g \preceq q$, and INSERTNODE then attaches g live, so every newly dead q has the live witness g . For a p that was already dead, the hypothesis gives a witness $\hat{g} \preceq p$; if $\hat{g}$ is still live we are done, and otherwise $\hat{g}$ was marked dead during this same insertion, which means $g \preceq \hat{g}$, so $g \preceq \hat{g} \preceq p$ by transitivity and g is a live witness for p . A rebuild retains exactly the live points and marks none dead, preserving the claim. The invariant holds at operation boundaries only: within one INSERT it is transiently violated, because MARKDOMINATED runs before g is attached.

Step 2: a retained dead point cannot change the answer. If CHECK returns TRUE, line 3 of Algorithm 2 tested $\text{POINT}(\nu) \preceq q$ at a live ν , so the witness is live. Conversely let some live p satisfy $p \preceq q$. A subtree at ν is skipped only when $\text{LO}(\nu)_i > q_i$ for some axis i ; since $\text{LO}(\nu)$ is the componentwise minimum over all points in that subtree, every point x there has $x_i \geq \text{LO}(\nu)_i > q_i$, so $x \not\preceq q$ and p is not among them. Thus p lies on an unpruned path and is tested. Dead points do enter the corners, but only conservatively: including them can only lower LO and raise HI, which makes the corner test succeed more often and therefore prunes less. Retaining them costs traversal, never correctness. The same argument covers MARKDOMINATED with HI in place of LO and the inequality reversed. Finally L is exactly the frontier: MARKDOMINATED removes from it precisely the vectors g dominates, so L stays an antichain, and by Step 1 no dead point is needed to represent it. $\square$

Theorem 1. We give part (ii) in full, then parts (i) and (iii).

Proof of (ii). Fix $d \geq 3$ and $n \geq 1$, and define the staircase instance

$$P = \{p^{(1)}, \dots, p^{(n)}\}, \quad p^{(i)} = (i, n+1-i, 1, \dots, 1),$$

with query $g = (n, n, 1, \dots, 1, 0)$: the first two coordinates descend, the rest are held at 1, and the query is 0 in the last coordinate.

P is an antichain, and stays one under any change to the last coordinate. For $i < j$, $p_1^{(i)} = i < j = p_1^{(j)}$ while $p_2^{(i)} = n+1-i > n+1-j = p_2^{(j)}$, so $p^{(i)}$ is strictly smaller in coordinate 1 and strictly larger in coordinate 2, making the two incomparable whatever their remaining coordinates are. Incomparability is thus witnessed by coordinates 1 and 2 alone, so replacing any single point's last coordinate by any value leaves an antichain. This is what the staircase is for, and it is where $d \geq 3$ enters: two coordinates are spent making the points pairwise incomparable and a third is left over to hide dominance in. At $d = 2$ nothing is left over, which is why part (i) holds instead.

On P there is no dominator, and no leading coordinate reveals it. Every $p \in P$ has $p_d = 1 > 0 = g_d$, so $p \not\preceq g$ and the correct answer on P is “no dominator”. Yet every $p \in P$ has $p_i \leq g_i$ for all $i < d$, so bounding the leading coordinates excludes nothing.

Any correct index must test every point. Suppose the index, run on P with query g , omits some $p^{(k)}$ from the interval it dominance-tests. Form P' from P by lowering only $p_d^{(k)}$ to 0. By the first paragraph P' is an n -point antichain, and its modified point $q = (k, n+1-k, 1, \dots, 1, 0)$ satisfies $q \preceq g$, so on P' the correct answer is “a dominator exists”. It remains to check that the index behaves identically on P and P' . Coordinate d is the least significant, and every $\prec$ -comparison the index performs is decided strictly earlier: the stored first coordinates are $1, \dots, n$, all distinct, so any comparison between two stored vectors is settled by coordinate 1; and a comparison of g against $p^{(i)}$ is settled by coordinate 1 when $i < n$, and by coordinate 2 when $i = n$, since $p_2^{(n)} = 1 \neq n = g_2$. No comparison reads coordinate d . The index therefore selects the same interval on P' as on P , again omits $p^{(k)}$, and finds no dominator among the points it does test, since those are unchanged and none dominates g . It answers “no dominator” on P' and is incorrect. Hence a correct index omits nothing and performs exactly n dominance tests. $\square$

The bound is tight in both directions on this family. On the total-order side n matches the trivial upper bound of testing every stored vector, so no better bound is possible; on the geometric side part (iii) gives 0 dominance tests after one corner test, so the gap on the staircase is n against $O(1)$, the largest it can be.

The argument also costs nothing to extend from deterministic to randomized indices, because the $n+1$ instances it constructs are mutually indistinguishable rather than adversarially chosen after the fact.

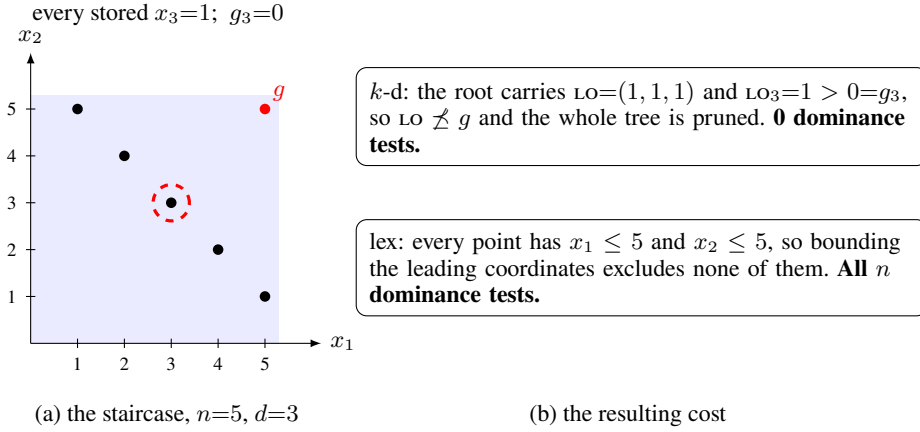


Figure 3: The construction of Theorem 1(ii), drawn at $n=5$, $d=3$. (a) The stored antichain is a staircase in the first two coordinates, so any two points are incomparable on those alone; the third coordinate is therefore free, and every point takes $x_3 = 1$. The query is $g = (5, 5, 0)$, which the shaded region shows every point clears in x_1 and x_2 , while no point dominates it because $x_3 = 1 > 0$. (b) The two indices see this differently: the corner-minimum settles it at the root, whereas the leading coordinates rule out nothing. The circled point carries the lower bound: lowering its x_3 to 0 leaves an antichain and creates a dominator, and no comparison in the order reads x_3 , so an index that skips that point cannot tell the two instances apart and answers one of them wrongly. The same holds for every point, so none may be skipped. Contrast Figure 4, which draws the more compact family of Remark 1.

Corollary 1 (Randomized indices). *Let $d \geq 3$ and $n \geq 1$. Let an index of Definition 1 form be allowed to randomize its choice of interval, and suppose it answers correctly with probability at least $2/3$ on every n -point antichain in $\mathbb{R}^d$. Then there is a distribution over n -point antichains and queries on which its expected number of dominance tests is at least $(2n - 1)/3$.*

Proof. Take the $n + 1$ instances P and $P^{(1)}, \dots, P^{(n)}$, where $P^{(k)}$ is P with $p_d^{(k)}$ lowered to 0; each is an n -point antichain by the first paragraph of the proof of (ii), and $P^{(k)}$ contains a dominator of g while P does not. Let the input distribution be uniform on these $n + 1$ instances, each paired with the query g . Fix any deterministic index. As shown above, no $\prec$ -comparison reads coordinate d , so the index selects the same interval on all $n + 1$ instances; let $S \subseteq \{1, \dots, n\}$ index the stored vectors it dominance-tests, so it performs $|S|$ tests on every instance. It answers correctly on P , and on $P^{(k)}$ exactly when $k \in S$, since for $k \notin S$ the only dominator of g is the one point it never tests. Its error probability is therefore $(n - |S|)/(n + 1)$, and requiring this to be at most $1/3$ forces $|S| \geq (2n - 1)/3$. Every deterministic index therefore performs at least $(2n - 1)/3$ dominance tests on this distribution, and by Yao's minimax principle the same lower bound holds for the expected number of dominance tests of any randomized index meeting the same error guarantee. $\square$

Proof of (i) and (iii). (i) The case $d = 2$. Store the antichain sorted by the first coordinate, as $p^{(1)}, \dots, p^{(n)}$. A two-dimensional antichain is strictly decreasing in the second coordinate along this order: if $p_1 < q_1$ and $p_2 \leq q_2$ then $p \preceq q$, contradicting incomparability. Fix a query g and let j be the largest index with $p_1^{(j)} \leq g_1$, found by binary search; if no such index exists then no stored vector clears the first coordinate and there is no dominator. Among $p^{(1)}, \dots, p^{(j)}$ the vector $p^{(j)}$ has the smallest second coordinate, since that coordinate decreases along the order. Hence a stored vector dominates g if and only if $p_2^{(j)} \leq g_2$, one dominance test. No index of any kind improves on this asymptotically.

(iii) The geometric index. On the staircase instance P of part (ii), the corner-minimum at the root satisfies $\text{LO}(\text{root})_d = 1 > 0 = g_d$, so $\text{LO}(\text{root}) \not\preceq g$ and Algorithm 2 returns FALSE at line 2 after a single corner test, performing no dominance test. For the general bound, the classical $O(dn^{1-1/d})$ result for orthant queries on a balanced k -d tree (Lee and Wong 1977) prunes a subtree when its axis-aligned region misses the query orthant. Our corner test prunes when the bounding box of the subtree's points misses it. The bounding box is contained in the region, so its near corner is componentwise no smaller than the region's, and the corner test succeeds whenever the region test does. Corner pruning is therefore at least as strong, and the bound carries over. $\square$

Remark 1 (The illustrative family of Figure 4). A more compact family shows the same n -versus-0 gap, and is the one we draw and replay. For $m \geq 1$ set the first $d - 1$ coordinates of each point to every tuple in $\{1, \dots, m\}^{d-1}$ and the last to $(d - 1)m - \sum_{i < d} x_i + 1$, giving an antichain of $n = m^{d-1}$ points on a hyperplane, with query $g = (m, \dots, m, 0)$. It is an antichain because raising any of the first $d - 1$ coordinates strictly lowers the last. Again no point dominates g , since every last coordinate is at least 1, and no leading coordinate excludes anything, so a lexicographic index tests all n points while the k -d tree prunes at the root. This family does not support the indistinguishability step of Theorem 1(ii): any vector dominating its

query is itself dominated by the all- m point of P , so no order-indistinguishable antichain with a hidden dominator exists. We therefore prove the theorem on the staircase and use this family for Figure 4 and Experiment 1, where both give exactly n AVL comparisons per CHECK against 0 for the k -d tree.

Empirical verification of Theorem 1. Both bounds of Theorem 1 are exact rather than asymptotic on the staircase, so they can be checked against the implementation to the digit. Table 3 replays the staircase through the AVL and k -d backends over four sizes and four dimensions. In all 16 configurations the AVL performs exactly n dominance comparisons per CHECK, as part (ii) requires, and the k -d tree performs exactly 0, pruning at the root on one corner test as part (iii) requires. The counts are deterministic, so these are equalities, not estimates.

n	AVL comparisons per CHECK				k -d comparisons per CHECK			
	$d=3$	$d=4$	$d=5$	$d=6$	$d=3$	$d=4$	$d=5$	$d=6$
256	256	256	256	256	0	0	0	0
1024	1024	1024	1024	1024	0	0	0	0
4096	4096	4096	4096	4096	0	0	0	0
16,384	16,384	16,384	16,384	16,384	0	0	0	0

Table 3: **Theorem 1 checked against the implementation.** The staircase instance of part (ii) is built at each (n, d) and replayed through both backends, with 50 identical queries $g = (n, n, 1, \dots, 1, 0)$ per configuration. The AVL total-order index tests every stored vector, matching the n of part (ii) exactly; the k -d index tests none, matching the 0 of part (iii). Comparison counts are exact and deterministic and so carry no interval.

Example cost vectors: $(1, 1, 5), (1, 2, 4), (1, 3, 3), (2, 1, 4), (2, 2, 3), (2, 3, 2), (3, 1, 3), (3, 2, 2), (3, 3, 1)$; query $g = (3, 3, 0)$.

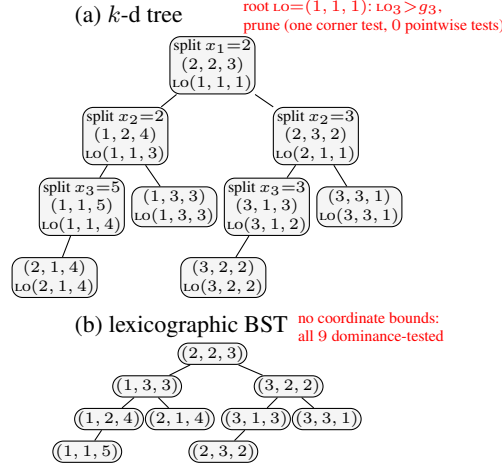


Figure 4: The illustrative family of Remark 1 at $m=3$ ($d=3, n=9$), query $g = (3, 3, 0)$. (a) The k -d tree root has corner-minimum $LO = (1, 1, 1)$. Since $LO_3 = 1 > 0 = g_3$, no stored point can lie in the query orthant and the whole tree is pruned by one corner test, with no dominance test. (b) The lexicographic BST bounds only the leading coordinates, which every point clears ($x_1 \leq 3$ and $x_2 \leq 3$), so it can rule out no subtree and dominance-tests all 9 points. Node labels give the stored point, and in (a) the split axis and LO ; the root of (a) splits on $x_1=2$, and the bulk build places one of the three points with $x_1=2$ in the left subtree (Section 3).

Proposition 2 (Single-coordinate orders). *Let $d \geq 3$, let $j \in \{1, \dots, d\}$, and let $\prec_j$ order cost vectors by coordinate j with ties broken lexicographically. For every $n \geq 1$ and every total-order index for $\prec_j$ that is correct on all n -point antichains in $\mathbb{R}^d$, there are an n -point antichain and a query on which it performs exactly n dominance tests.*

Proof. Since $d \geq 3$ we may pick coordinates b and k distinct from each other and from j . Run the staircase in coordinates j and b and hold every other coordinate, including k , at 1:

$$p_j^{(i)} = i, \quad p_b^{(i)} = n + 1 - i, \quad p_\ell^{(i)} = 1 \quad (\ell \notin \{j, b\}),$$

for $i = 1, \dots, n$; and query with $g_j = n + 1, g_b = n, g_k = 0$, and $g_\ell = 1$ otherwise.

As in Theorem 1(ii), incomparability is witnessed by coordinates j and b alone, so lowering any one point’s coordinate k to 0 leaves an antichain and creates a vector dominating g ; and every p has $p_k = 1 > 0 = g_k$, so on P there is no dominator, while $p_\ell \leq g_\ell$ for every $\ell \neq k$.

Setting $g_j = n + 1$ rather than n is what makes the argument independent of the tie-break. The stored vectors take the n distinct values $1, \dots, n$ in coordinate j , so every comparison between two stored vectors is settled by coordinate j ; and $g_j = n + 1$ exceeds every stored value, so every comparison of g against a stored vector is settled by coordinate j as well. The tie-break never runs, and no $\prec_j$ -comparison reads coordinate k . The indistinguishability argument of Theorem 1(ii) then applies verbatim: omitting any point lets an adversary hide a dominator there, so a correct index tests all n . $\square$

B The Average-Case Arguments in Full

This appendix expands Section 4.2. Neither argument is a theorem about multi-objective A^* , and we state precisely where each stops applying.

How large is a frontier? Model the costs of the r labels reaching a vertex as r independent draws from a fixed continuous distribution on $\mathbb{R}^d$ whose d components are themselves independent. The frontier is then the set of *minima* of the sample: a vector survives only if no other draw is smaller in all d coordinates simultaneously. Bentley et al. (1978) show that the expected number of minima satisfies

$$\mathbb{E}[n] \sim \frac{(\ln r)^{d-1}}{(d-1)!} \quad \text{as } r \rightarrow \infty \text{ for fixed } d,$$

where $\sim$ means the ratio of the two sides tends to 1; equivalently $\mathbb{E}[n] = \Theta((\ln r)^{d-1})$ at fixed d , the factorial being absorbed into the constant. Note the two independence assumptions are distinct and both needed: independence *across* the r draws, and independence *among* the d components of each draw. The frontier therefore grows poly-logarithmically in the number of labels generated at the vertex rather than linearly, because most labels are dominated on arrival.

Component independence is what drives this, and real objectives are correlated. A pairwise correlation ρ makes a vector that is cheap on one objective cheap on the others, so fewer vectors survive and fronts shrink, while decorrelation creates genuine trade-offs and large fronts. Correlation thus acts like a reduction in the effective number of independent objectives in the bound, which is why ρ is the parameter our generators expose (Section 5 and Appendix C): sweeping it sweeps the front-size regime that Section 4.3 identifies as the thing the speedup tracks.

What does a query cost on such a frontier? On a thin, spread-out antichain most subtrees fail the corner test, so an orthant-emptiness query visits few nodes and its comparisons per CHECK grow far more slowly than the linear scan’s $\Theta(n)$. Quantifying “few” is where the argument stops being rigorous, for two separate reasons. The closest available average-case result is the partial-match analysis of Flajolet and Puech (1986), which counts node visits for queries that fix some coordinates exactly and leave the rest free; an orthant query constrains *every* coordinate by an inequality instead, so it is not a partial-match query and the analysis does not directly apply. And that analysis assumes the stored points are an order-independent random sample, whereas in multi-objective A^* the labels are correlated by construction and their insertion order is structured by the lexicographic pop order, which is exactly the order the frontier is built in. We therefore claim no average-case bound. What survives is a qualitative, testable prediction: comparisons per CHECK should grow as a sub-linear power of n , and should sit below the $n^{1-1/d}$ worst case. Experiment 2 measures that exponent per dimension, and Figure 2 reports it.

C Geometric Cost Generation on Real Topologies

Goal. We want objectives that resemble real multi-objective routing, where each criterion (travel time under local congestion, a toll or emission zone, terrain) is a *spatially coherent* attribute that varies smoothly across the map and trades off against the others, and whose mutual correlation we can dial to sweep the front-size regime. The common synthetic alternative, uniform i.i.d. random edge weights, lacks this coherence: neighbouring roads get unrelated costs, which is geographically implausible and, as Experiment 4 shows, *overstates* Pareto-front sizes relative to coherent costs. Our model therefore produces geographically smooth attributes with a tunable correlation ρ , and we report the random model alongside as a control rather than as the primary instance. The construction below is fully seeded and deterministic; each vertex’s objective values come from scalar fields over the graph, so the costs reflect the map’s geometry rather than per-edge noise.

Landmark fields. The PACE conversions of the DIMACS topologies carry no vertex coordinates, but graph distance to a set of landmark vertices approximates geographic position, the principle underlying ALT (A^* , Landmarks and Triangle inequality) routing. We select $L = \max(8, 2M)$ landmarks by farthest-point sampling: starting from a seeded vertex, each new landmark is the vertex maximizing the hop distance to the current landmark set. From each landmark $\ell \in \{1, \dots, L\}$ a breadth-first search yields a hop-distance field $\delta_\ell : V \rightarrow [0, 1]$, rescaled by its maximum so that the farthest vertex maps to 1. The L sweeps are linear in the graph size.

Objective fields. A purely distance-like field tracks position monotonically and so yields near-identical objectives on a near-planar road map (tiny fronts). We therefore build each objective as a multi-frequency, Perlin-like field over the landmark distances, which produces smooth attributes that vary across the map without merely tracking position. We draw M amplitude

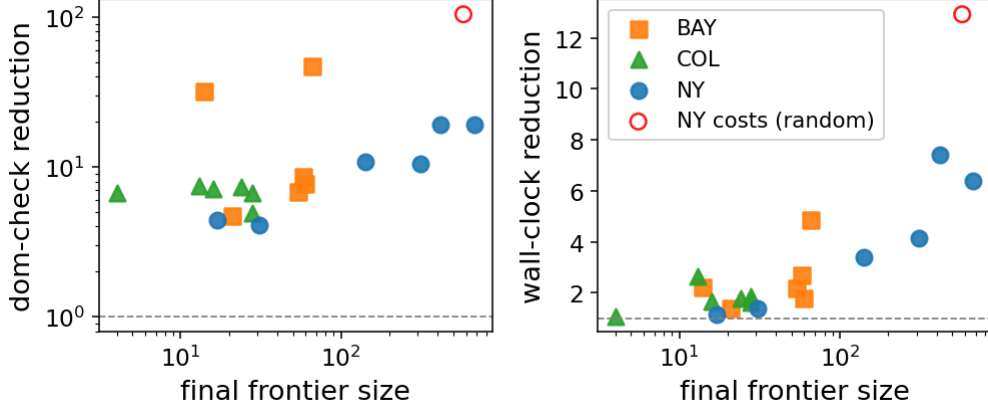


Figure 5: **Real road replay study** over 19 captured streams from three DIMACS maps. *Left*: the AVL/ k -d reduction in check comparisons against per-vertex frontier size n , reaching $104\times$; *right*: the corresponding wall-clock reduction, reaching $12.9\times$. Both increase with n overall rather than with map size. Open red markers are the random i.i.d. cost model, filled markers the geometric one; dashed lines mark parity, below which the k -d index would be the slower of the two.

vectors $a_1, \dots, a_M \in \mathbb{R}^L$ from a seeded Gaussian and orthogonalize them by Gram–Schmidt, and draw for each objective $k \in \{1, \dots, M\}$ and each landmark ℓ a seeded integer frequency $f_{k,\ell} \in \{1, \dots, 6\}$ and a phase $\psi_{k,\ell}$ uniform on $[0, 2\pi)$. Writing $a_{k,\ell}$ for the ℓ -th entry of a_k , the raw objective field at vertex v is

$$\tilde{\phi}_k(v) = \sum_{\ell=1}^L a_{k,\ell} \sin(2\pi f_{k,\ell} \delta_\ell(v) + \psi_{k,\ell}),$$

which we then rescale affinely so that its minimum over V is 0 and its maximum is 1. A correlation knob blends each objective toward a single shared distance-like field $\phi_0 = \frac{1}{L} \sum_{\ell} \delta_\ell$, the unweighted mean of the landmark fields, giving $\phi_k = \rho \phi_0 + (1 - \rho) \tilde{\phi}_k$; at $\rho=1$ all objectives coincide, and at $\rho=0$ each follows its own field. Edge $e = (u, v)$ then receives cost

$$c_k(e) = 1 + \lfloor 64 \cdot \frac{1}{2} (\phi_k(u) + \phi_k(v)) \rfloor \in \{1, \dots, 65\},$$

in objective k , where $\lfloor \cdot \rfloor$ denotes rounding to the nearest integer, so every cost is a positive integer as MO-SPP requires. A path therefore accumulates a spatially smooth attribute along its route, a congestion, toll or terrain band, and different objectives favor routes through different regions, so the trade-offs are genuine rather than degenerate.

Disclosure and control. Because ρ is a blending weight rather than a correlation, we also report the correlation it actually achieves, measured as the mean absolute pairwise Pearson correlation over the per-edge objective vectors. On grid topologies it ranges from 0.99 at $\rho=0.9$ down to 0.53 at $\rho=0.5$, and on New York it remains as high as 0.73 even at $\rho=0$. That residual correlation is the point: the fields stay spatially coherent, as real geographic attributes are, so the objectives end up realistically rather than artificially decorrelated. As a control, the experiments also report a random i.i.d. cost model in which each objective is an independent per-edge integer draw under the same ρ blend, which isolates the effect of spatial coherence from that of decorrelation alone.

Instance	n	k -d (ours)	t (ms)	chk↓	upd↓	wall↓
COL $M5 \rho.5$	24		366 ± 71	7.4	4.7	1.79
COL $M5 \rho.7$	28		78 ± 4	6.7	3.3	1.88
BAY $M5 \rho.6$	54		26.5 ± 1.2	6.8	5.7	2.17
BAY $M5 \rho.5$	60		56.5 ± 8.4	7.8	3.2	1.77
NY $M4 \rho.5$	142		249 ± 7	10.8	6.2	3.39
NY $M5 \rho.65$	418		842 ± 74	19.1	18.4	7.42
NY $M5 \rho.6$	678		1191 ± 19	19.1	22.3	6.37
NY $M5 \rho.7$ (random)	578		831 ± 47	104	14.8	12.9

Table 4: **Real road replay** (the Experiment-1 protocol applied to captured road streams). Each row replays the busiest vertex’s fixed operation stream through the AVL and k -d indices; n is that vertex’s peak frontier size. Columns give the k -d replay wall-clock (mean over repeated replays with a 95% confidence interval) and the AVL/ k -d reduction in check comparisons (chk↓), update comparisons (upd↓) and replay wall-clock (wall↓); comparison counts are exact and deterministic and so carry no interval. Every reduction exceeds 1. Rows are ordered by n and use the geometric cost model except the last, which uses random i.i.d. costs. These eight rows are selected to span the range of n ; the full sweep of 19 streams, including two Bay $M=4$ streams whose reductions reach $32\times$ and $46.9\times$ at small n , appears in Figure 5.

D Dynamic Frontier Example

To illustrate the dynamic k -d tree operations, consider a 3-objective search, so the index tracks the $d = 2$ trailing objectives and each stored vector is a pair (x_1, x_2) . Suppose the tree (Figure 6a) holds four live vectors $A(5, 5)$, $B(3, 7)$, $C(8, 2)$ and $D(6, 4)$, together with one already *dead* node $E(4, 8)$, a vector that an earlier insertion of B dominated. The live set is an antichain, as a frontier must be: sorted by x_1 it reads $(3, 7)$, $(5, 5)$, $(6, 4)$, $(8, 2)$, strictly decreasing in x_2 , which is exactly the form a two-dimensional antichain takes. Structurally, A is the root and splits on x_1 ; its left child is B , which splits on x_2 and has E as its right child; its right child is C , which splits on x_2 and has D as its right child. The corners follow from the subtree memberships, with dead E included: $\text{LO}(A) = (3, 2)$ and $\text{HI}(A) = (8, 8)$; $\text{LO}(B) = (3, 7)$ and $\text{HI}(B) = (4, 8)$; $\text{LO}(C) = (6, 2)$ and $\text{HI}(C) = (8, 4)$.

Pass 1: Check. A new label arrives with trailing objectives $g = (4, 4)$. Algorithm 2 descends from the root. At A the corner-minimum is $\text{LO}(A) = (3, 2)$, and neither $3 > 4$ nor $2 > 4$, so the subtree is not pruned; the point test gives $A(5, 5) \not\preceq (4, 4)$ because $5 > 4$ in the first coordinate. Descending left to B we have $\text{LO}(B)_2 = 7 > 4$, so the whole left subtree, B and E together, is corner-pruned without inspecting either point. Descending right to C we have $\text{LO}(C)_1 = 6 > 4$, so the right subtree is pruned as well. No live dominator exists, and g is accepted after one point test and two corner tests, in place of the four point tests a scan of the frontier would need.

Pass 2: Update (MarkDominated). Algorithm 3 now runs. Its `MARKDOMINATED` helper sweeps the upper orthant $[g, \infty)$, using the corner-maxima, to tombstone every live vector that g dominates. At A , $\text{HI}(A) = (8, 8)$ has no component below g , so the node is visited, and $g \preceq A(5, 5)$ since $4 \leq 5$ in both coordinates: the root A is marked *dead*. At B , $\text{HI}(B) = (4, 8)$ again has no component below g , so the sweep descends; $g \not\preceq B(3, 7)$ because $4 > 3$, so B stays live. Visiting its child E , we find $g \preceq E(4, 8)$, but E is already dead, so the point-level test is skipped and nothing changes. At C , $\text{HI}(C) = (8, 4)$ is not below g either; $g \not\preceq C(8, 2)$ because $4 > 2$ in the second coordinate, so C stays live, and at its child $g \preceq D(6, 4)$, so D is marked *dead*. The live set is now $\{B(3, 7), C(8, 2)\}$, and adding g restores an antichain $\{(3, 7), (4, 4), (8, 2)\}$.

Pass 3: Update (InsertNode). Finally `INSERTNODE` routes g to a leaf (Figure 6b). At the root A , which splits on axis 0, we compare $g_1 = 4$ against $\text{POINT}(A)_1 = 5$; since $4 < 5$ the descent goes left, *through the node just marked dead*, which is precisely why dead nodes are retained rather than removed. At B , splitting on axis 1, we compare $g_2 = 4$ against $\text{POINT}(B)_2 = 7$ and again go left, finding a `NIL` child, so $g(4, 4)$ is attached there as a new live leaf. As the recursion unwinds, each node on the path refreshes its corners: B 's corner-minimum widens to $\text{LO}(B) = \min((3, 7), (4, 4)) = (3, 4)$, while $\text{LO}(A) = (3, 2)$ and the corner-maxima are already wide enough to contain g and do not move. The update completes in place, in time proportional to the depth, with two nodes tombstoned and none physically removed.

E Implementation and Measurement Details

All backends share one C++ interface (`CHECK/UPDATE`) built upon the original EMOA* codebase¹. Road network instances are the PACE 2016 conversions of the 9th DIMACS Implementation Challenge road graphs². The codebase is compiled with g++ 13.2.0 (MinGW-w64, UCRT runtime) via CMake 3.29.2 at `-O2`; every wall-clock figure in the paper uses `-O2` (the shipped default `-O0` build is unoptimized and substantially slower). Dominance comparisons are tallied by a single instrumented predicate that all backends call, so the counts are exact and compiler-independent. The k -d backends use a contiguous array store and an arena node allocator. Wall-clock is measured single-threaded on one core of a consumer laptop (Intel Core i7-11800H, 8 cores at 2.3 GHz, 24 MB L3 cache, 64 GB RAM, Windows 11); no GPU is used. Each reported mean is over K repetitions, of a replay in Tables 1 and 4 and of a full search in Table 2, with a 95% confidence interval; K is given per row in Table 2 and in the caption of Table 1. All randomness (cost generation, landmark and frequency draws, and instance sampling) is seeded and propagated, so every instance is reproducible. The index's structural parameters are fixed a priori rather than tuned: the rebuild doubling factor ($2\times$) and its slack ($s=8$), the landmark count $L=\max(8, 2M)$, and the Perlin frequency range $\{1, \dots, 6\}$ are held constant across all runs, while M and ρ are swept as experimental conditions, not selected for performance.

Benchmark instances. For the real robot-arm end-to-end test (Table 2) we evaluate the `panda-RRG_8` instance, whose road-map is built by a Rapidly-exploring Random Graph (RRG). Seven cost files are loaded for this query in the benchmark, but two of them are identically zero, so only five objectives are active and we report that active count as $M=5$ rather than the nominal $M=7$.

Rebuild trigger. The trigger of Section 3 compares the physical node count against $2 \text{live}_{\text{last}} + s$, a snapshot taken at the previous rebuild. Comparing instead against the *current* live count would additionally cap the dead fraction at one half, and still amortize to $O(\log n)$ per update: at such a trigger the dead nodes outnumber the live ones, so the $O(m \log m)$ rebuild can be charged to the deaths that caused it, and each node is inserted once and dies at most once. We keep the snapshot form because it rebuilds less often, and the space bound it gives is what the analysis in Section 4.3 uses.

The cost of maintaining balance. The $O(dn^{1-1/d})$ bound of Section 4.1 is stated for a balanced k -d tree, and we can supply one: a second backend keeps the tree median-balanced by rebuilding it from the live set on every update, so the invariant holds

¹https://github.com/rap-lab-org/public_emoa

²<https://github.com/ben-strasser/road-graphs-pace16>

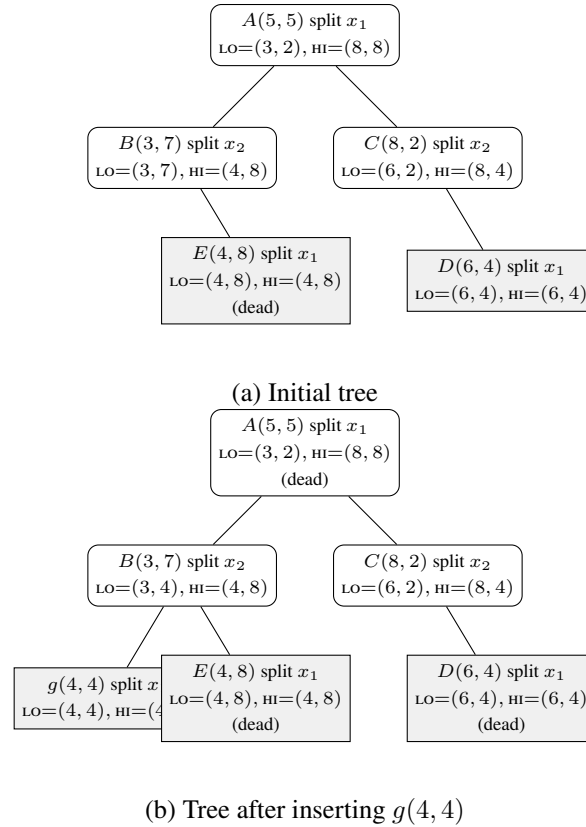


Figure 6: The dynamic k -d tree before and after inserting $g = (4, 4)$, as traced in Appendix D. The four live vectors of (a) form an antichain, and E is a node left dead by an earlier insertion. Inserting g tombstones both A and D , the two live vectors it dominates, leaves the already-dead E untouched, and attaches g as a new live leaf under B , widening only $\text{LO}(B)$ from $(3, 7)$ to $(3, 4)$. The descent to that leaf passes through the just-tombstoned root A , which is why dead nodes are retained: they no longer answer dominance tests but still route insertions and still contribute to the corners.

at every step. Table 5 replays each captured grid stream through all five backends, so every row reports one identical operation sequence, and the entries are comparisons per `CHECK`.

Two things separate. Balancing barely affects pruning, costing $1.05\text{--}1.53\times$ more comparisons per `CHECK` than the incremental form, and the balanced variant still does $3.6\text{--}6.9\times$ fewer than the ND-tree and $3.7\text{--}84\times$ fewer than the AVL, so the separation of Section 4.1 does not depend on which variant is used. The reason balance is close to irrelevant here is that Algorithm 2 never consults a node’s split coordinate: it prunes when a subtree’s corner-minimum cannot dominate the query and otherwise descends into both children, so query cost tracks how tightly the corners enclose their points rather than the depth that median splitting optimizes.

What balancing does cost is update time. Rebuilding on every update is $O(m \log m)$ and recomputes the non-dominated set by scanning, which puts the balanced variant’s update comparisons $7.7\text{--}169\times$ above the incremental form’s and makes it slower in wall-clock than every baseline from $M=5$ on. Lazy deletion is therefore not an incidental engineering choice: it is what turns the pruning advantage into a runtime one. A self-balancing dynamization (Cai, Xu, and Zhang 2021; Brown 2025) would in principle give the guarantee at an update cost closer to the incremental form’s; we have not measured one.

Memory footprint. Each k -d node stores its point plus two corner vectors (LO, HI) of $d = M - 1$ components, about $3\times$ the per-node vector storage of a plain lexicographic node, and lazy deletion tolerates up to a $2\times$ tombstone factor before a rebuild; the frontier structure is thus a small constant factor larger than a balanced binary search tree (BBST) or a list. This constant sits on an already-tiny object: a frontier of n live points uses $O(n)$ nodes with n at most a few thousand in our runs (Section 5), so its footprint is well under a megabyte and is dominated by the graph (up to 435,666 vertices with their adjacency) and the OPEN list (millions of labels), which set peak process memory. The extra corner vectors therefore change the total footprint negligibly while enabling the corner pruning the method depends on.

M	d	comparisons per CHECK				
		Linear	AVL	ND-tree	k -d incremental	k -d balanced
3	2	10.3	4.3	4.2	0.76	1.16
4	3	181.2	92.1	20.7	3.30	4.54
5	4	1242.4	677.2	44.8	7.36	8.51
6	5	3238.7	1319.8	107.6	14.96	15.65

Table 5: **Balanced against incremental, and both against the baselines.** Each row replays one captured grid stream through all five backends, so the comparison is over an identical operation sequence; counts are exact and deterministic. k -d *balanced* rebuilds a median-balanced tree from the live set on every update, so it satisfies the precondition of the $O(dn^{1-1/d})$ bound at every step, whereas k -d *incremental* is the structure of Section 3. Balancing costs 1.05 – $1.53\times$ in comparisons per CHECK and leaves the advantage over every baseline intact. These streams are the full captures; the $M=5$ and $M=6$ rows are therefore longer than the correspondingly labelled rows of Table 1, which replay time-capped prefixes, so the two tables agree at $M=3, 4$ and not above.

F Sensitivity to Objective Ordering

The frontier indices operate on the $d = M - 1$ trailing objectives, listed in an arbitrary order, and several of them consult that order internally: the AVL keys on the *lexicographic* order of the objectives, a lexicographically-sorted linear scan terminates on the *leading* one, and the k -d tree cycles its split axis through them. Salzman et al. (2023) single out this objective ordering as an open question. We quantify its effect by replaying each captured stream of Section 5 under all $d!$ permutations of the objective coordinates, holding the query stream fixed and recounting dominance comparisons; the counts are exact, so each ordering is run once. To make the mechanism explicit we add a *sorted linear* scan (a lexicographically ordered list with early termination) alongside the paper’s unsorted linear scan.

Two backends are *exactly* invariant across orderings (Tables 6–7): the unsorted linear scan, whose insertion-order scan never consults a coordinate, and the ND-tree, whose partitioning uses coordinate-symmetric Euclidean distance rather than any axis. Among the order-dependent indices, the lexicographic AVL varies by up to $2.2\times$ on the grid and $4.7\times$ on road streams, and the sorted linear scan by up to $12.1\times$: leaning entirely on the leading objective, it is fast when that objective is well chosen and no better than a full scan when it is not. The k -d tree is far more robust, varying by at most $1.82\times$, because its corner test prunes on *all* d coordinates rather than a single leading one, so no ordering collapses it to the total-order regime. The strongest statement the data supports is that the k -d tree’s *worst* ordering uses fewer comparisons per CHECK than the *best* ordering of every baseline, on every instance: in Table 6 the k -d “worst” column lies below every other “best” column, and the same holds on all road streams, whose absolute counts we omit for space and report only as ratios in Table 7. The geometric advantage is therefore not an artifact of a fortunate objective order, which is consistent with Proposition 2: the total-order indices are order-sensitive because they lean on one coordinate, and no ordering escapes their worst case.

Sorted vs. unsorted linear, and the choice of baseline. The main comparison uses the *unsorted* linear scan, the frontier list of the NAMOA*/EMOA* reference implementations. A lexicographically-sorted scan does strictly fewer dominance comparisons at the natural ordering (Table 8), because early termination skips the tail of the list, but it pays an $O(n)$ sorted insertion on every update. The two therefore do *not* dominate one another in wall-clock: the sorted scan is faster only once the frontier is large enough for the comparison savings to outweigh the insertion cost ($1.23\times$ at $n=1992$ on the grid), and is tied on the small per-vertex frontiers of the road streams, where it can even trail the unsorted scan despite doing fewer comparisons. Neither is a uniformly stronger linear baseline, which is why we report the standard unsorted list. The k -d tree, by contrast, does far fewer comparisons than *both* at every front size (Table 8) and, on the same synthetic streams, is by a wide margin the fastest backend in wall-clock (Table 1); the sorted/unsorted question therefore concerns only which linear scan serves as the baseline, not the standing of the geometric index.

M	Backend	best	worst	mean $\pm$ std
3	Linear (unsorted)	10.3	10.3	10.3 ± 0.0
	Sorted linear	3.03	7.14	5.09 ± 2.91
	AVL (lex)	4.34	7.85	6.09 ± 2.48
	k -d (ours)	0.76	0.77	0.77 ± 0.01
	ND-tree	4.15	4.15	4.15 ± 0.0
4	Linear (unsorted)	181	181	181 ± 0.0
	Sorted linear	86.4	110	96.6 ± 11.0
	AVL (lex)	84.9	111	96.0 ± 11.8
	k -d (ours)	3.30	3.88	3.54 ± 0.24
	ND-tree	20.7	20.7	20.7 ± 0.0
5	Linear (unsorted)	695	695	695 ± 0.0
	Sorted linear	180	390	316 ± 82.4
	AVL (lex)	177	387	312 ± 81.8
	k -d (ours)	5.19	5.93	5.53 ± 0.24
	ND-tree	30.8	30.8	30.8 ± 0.0
6	Linear (unsorted)	1847	1847	1847 ± 0.0
	Sorted linear	745	1276	989 ± 185
	AVL (lex)	726	1265	974 ± 186
	k -d (ours)	11.1	14.5	12.6 ± 0.85
	ND-tree	73.8	73.8	73.8 ± 0.0

Table 6: Objective-ordering sensitivity on the synthetic grid (the streams of Table 1), replayed under all $d!$ coordinate orderings. Comparisons per CHECK; best/worst/mean over orderings. The total-order indices swing widely while the k -d tree stays nearly flat, and its *worst* ordering still uses fewer comparisons than any baseline’s *best*.

Road instance	Linear	Sorted	AVL	k -d (ours)	ND
COL $M4 \rho.6$	1.00	7.92	1.08	1.82	1.00
NY $M4 \rho.7$	1.00	4.15	1.73	1.08	1.00
BAY $M4 \rho.5$	1.00	12.1	4.69	1.09	1.00
BAY $M5 \rho.7$	1.00	1.54	1.37	1.44	1.00
COL $M5 \rho.7$	1.00	3.31	2.26	1.34	1.00
NY $M5 \rho.6$	1.00	2.50	2.35	1.26	1.00
NY $M5 \rho.7$ (rand)	1.00	4.70	4.11	1.21	1.00

Table 7: Objective-ordering sensitivity on real road streams: the *worst/best* ratio of dominance comparisons per CHECK over all $d!$ orderings. The unsorted linear scan and the ND-tree are exactly invariant (1.00); the total-order indices swing most (sorted linear up to $12\times$, AVL up to $4.7\times$), while the k -d tree stays within $1.8\times$. On every instance the k -d tree’s worst ordering still beats each baseline’s best in absolute comparisons.

Instance (front n)	comparisons / CHECK			
	Lin	Sort	k -d (ours)	$t_{\text{Lin}}/t_{\text{Sort}}$
grid $M=5$ ($n=1992$)	695	359	5.5	$1.23\times$
grid $M=4$ ($n=485$)	181	93	3.3	$1.20\times$
COL $M5 \rho.7$ ($n=28$)	21	14	2.3	$1.01\times$
NY $M4 \rho.7$ ($n=17$)	5.0	2.2	0.7	$1.10\times$

Table 8: Unsorted vs. lexicographically-sorted linear scan, at the natural objective ordering. Comparisons per CHECK are exact and deterministic; the two grid rows match the Linear and k -d entries of Table 1 at $M=5$ and $M=4$. The road rows are streams from the Experiment-4 sweep and are not among the eight rows listed in Table 4. The last column is the sorted scan’s wall-clock speedup over the unsorted scan ($t_{\text{Lin}}/t_{\text{Sort}}$, single-core; >1 means sorted is faster). The sorted scan does fewer comparisons throughout, yet is faster only at large frontier size n and tied ($\approx 1\times$) at small n , so neither dominates the other. The k -d tree does far fewer comparisons than both, and on the synthetic streams is the fastest backend in wall-clock by a wide margin (Table 1).